Project Proposal: Multi-Tone Communication Assistant for Zulip

Course : ECE-GY9183 Professor: Fraida Fund

Team Members:

Sudarshan Rao Allu(sa9876)

Hardik Amarwani (ha3290)

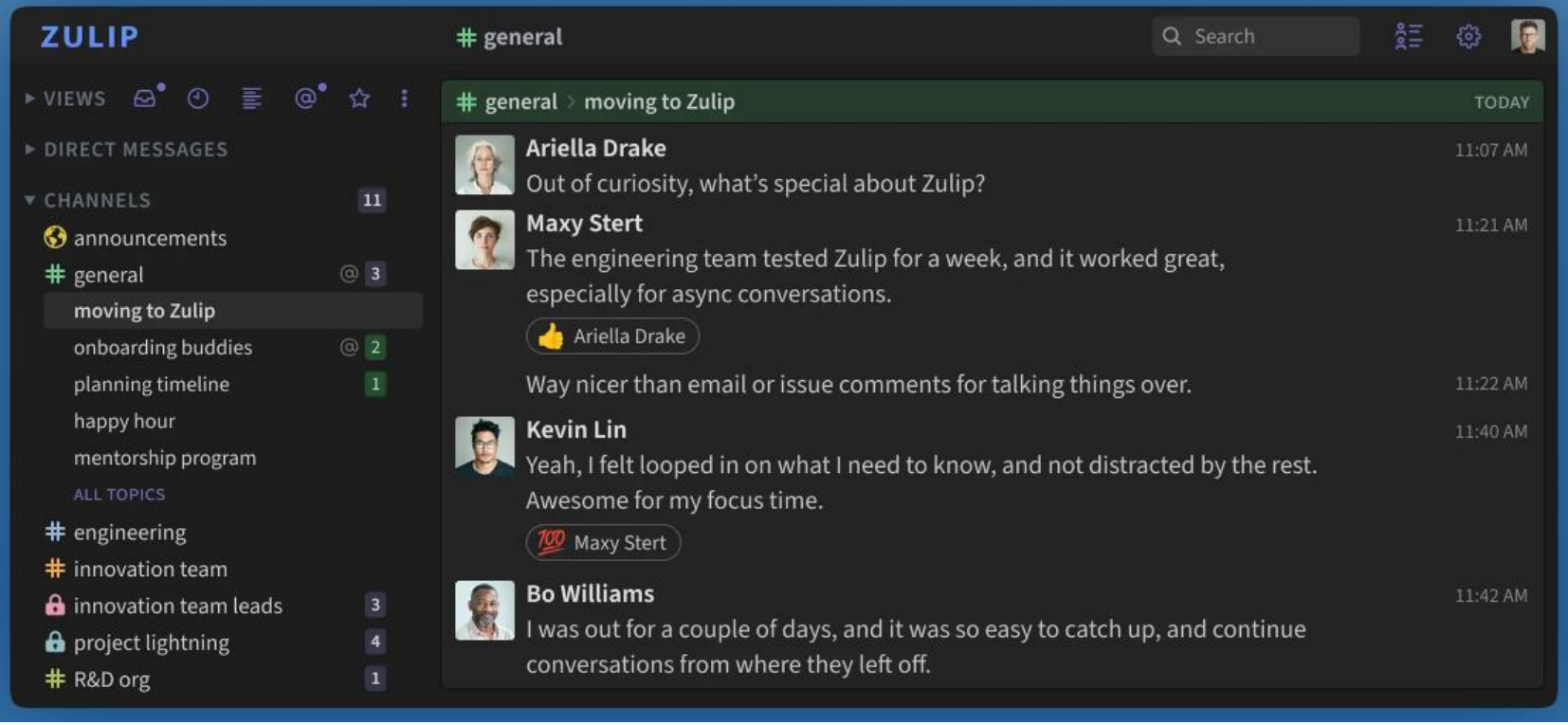
Rithwik Amjala(sa9880)

Deeksha Ravishankar(dr3956)

Public-Facing Service Overview

- **Deployment:** Self-hosted team collaboration platform using Zulip on Chameleon Cloud
- **Target Users:**
  - University project teams
  - Research labs
  - Small engineering teams (50–200 users)
- **Primary Users:** Students, teaching assistants, research collaborators, early-stage startups
- **Communication Context:**
  - Stream messages, topic-based threads, and direct messages
  - Fast-paced, task-oriented, often informal or emotionally reactive
  - Written under time pressure → tone easily misinterpreted
- **Current Workflow & Pain Points:**
  - Users type and send messages; misunderstandings require manual clarification
  - Challenges: Unintentionally harsh messages, tone mismatch, difficulty for non-native speakers
  - Manual rewriting is slow and cognitively demanding

Organized chat for distributed teams



Multi-Tone Assistant Design

- **Purpose:** Improve message tone without removing human control
- **User Flow:**
- Draft message → Click “Improve Tone”
- System suggests 3 rewrites: Formal, Friendly, Neutral
- User selects, edits, or ignores → Message sent manually
- **ML Architecture:**
- **Tone Classifier (DistilBERT):** Predicts tone, evaluates outputs, provides probability distribution for ranking and monitoring
- **Controlled LLM Generator:** Produces semantically equivalent rewrites for each target tone

External Datasets (GYAFC)

- **Lineage**
  GYAFC was created by Grammarly researchers using data from Yahoo Answers. Informal sentences from two domains (Entertainment & Music, Family & Relationships) were professionally rewritten into formal versions by trained human annotators. The dataset was designed specifically for formality style transfer research.
- **Structure**
  Parallel corpus of informal–formal sentence pairs:

Source sentence (informal)

Target sentence (formal rewrite)

- **Example**
- Informal: “i cant believe this lol”
  Formal: “I find this difficult to believe.”
- **Relevance to Zulip Tone Assistant**
  Zulip is used in academic and professional settings where tone appropriateness is critical. GYAFC directly supports the “Formal” tone class by teaching the model to transform casual messages into workplace-appropriate language while preserving semantic meaning.

External Dataset - Stanford Politeness Corpus

- **Lineage**
  Collected by Stanford researchers from StackExchange and Wikipedia discussions. Requests were annotated by multiple human raters to measure politeness in online collaboration.

Structure

- Request sentence
- Politeness score or label (crowd-sourced)

Example

- “Fix this.”
- “Could you please take a look at this when you have a chance?”
- The second receives a higher politeness score due to softer phrasing.

Relevance to Zulip
Zulip conversations often involve task coordination and requests. This dataset helps train the classifier to distinguish blunt vs. polite phrasing, supporting both “Formal” and “Friendly” tone categories.

Training Design

Multi-Tone Assistant: Training Architecture

- Hybrid ML design:
    - **Tone Classification Model** – DistilBERT fine-tuned for 3-class tone prediction (Formal, Friendly, Empathetic, Concise)
    - **Controlled Tone Generator** – Instruction-tuned Transformer LLM for message rewriting
- Goals: Accurate tone suggestions, fluent output, continuous improvement
- Deployed on Chameleon with privacy and latency constraints
- Includes training and retraining strategy for classifier and generator

Visual Suggestion:

- Flow diagram:

User Message → Classifier → Generator → User Selection → Feedback Loop → Retraining

Tone Classification Model

Tone Classifier – DistilBERT

- **Purpose:** Predict tone, evaluate generator outputs, monitor drift
- **Training / Retraining:**
    - Fine-tuned DistilBERT with 3-class classification head
    - Trained on GYAFC , Stanford Politeness Corpus
    - Cross-entropy loss, weighted class balancing, early stopping on validation F1
    - Retraining weekly or triggered by acceptance rate drop, tone drift, or confidence decline
    - Only high-confidence feedback included; undo or inconsistent edits excluded
- **Input Features:** Tokenized message text (max 128 tokens), optional message type (stream/direct), excluding PII
- **Outputs:** Probability distribution over 3 tones, confidence score, predicted dominant tone
- Feedback used for retraining and drift monitoring
- **Visual Suggestion:**
- Classifier flow:

Input → DistilBERT → 3-class probability + confidence → Feedback for retraining

Tone Generator

Tone Generator – Instruction-Tuned LLM

- **Purpose**: Rewrite messages in Formal, Friendly, or Neutral tones
- **Training / Fine-Tuning:**
    - Prompt-controlled generation initially
    - Token-level cross-entropy loss; future reinforcement-style updates using acceptance rate
- **Constraints:** Latency < 600ms, no automated sending, privacy maintained
- **Input Features:** Raw user message, explicit tone instruction, optional context of prior messages
- **Outputs:** Three rewritten variants, validated by classifier, with confidence scores and offensive content flagged

Visual Suggestion:

- Generator flow:

User message + tone instruction → LLM → 3 variants → Classifier validation → User → Optional fine-tuning

Operational Requirements

Serving Requirements

User Assumptions

- ~200 active users
- Peak: 5 messages/min/user
- Worst case: ~1000 requests/min
- ≈ **17 requests/sec peak load**

Latency Target (p95 < 800 ms)

- Classifier (CPU): < 100 ms
- Generator (GPU): < 600 ms
- API + validation: < 100 ms

Provisioning Strategy

- Burst-aware design
- Feature is user-triggered (not automatic)
- Designed for peak academic usage (deadlines)

Serving Architecture

Inference Pipeline

- User Draft
  → Tone Classifier (DistilBERT)
  → Controlled Generator (LLM)
  → Rank & Validate
  → Return 3 Variants
  → Log Interaction

Architecture Design

- Classifier: Stateless CPU microservice
- Generator: Dedicated GPU service
- Request batching for burst handling
- Autoscaling if CPU > 70%

Monitoring

- p95 latency
- Error rate
- Acceptance rate

Model Output → System Impact

System returns:

- 3 tone-conditioned rewrites
  - Formal
  - Friendly
  - Neutral
- Confidence scores
- User selects variant → replaces draft → manual send.

Measured Outcomes

- Suggestion acceptance rate ≥ 35%
- Undo rate < 5%
- Improved tone alignment in professional streams

Feedback Loop

- Log original + variants + selection + edits
- Feeds retraining pipeline
- Enables drift detection + continuous improvement

Data Flow (Real-Time Pipeline)

Input

- User message + minimal metadata
- Selected tone (formal / friendly/ Neutral)

Processing

- Validation and normalization
- Optional limited context
- LLM generates tone-aligned rewrite
- DistilBERT classifier evaluates tone consistency

Output

- Grammar and meaning checks
- Suggestion returned to user
- Privacy-preserving logging for monitoring

Training Data Pipeline

Data Sources

- Curated external corpora
- Validated user feedback logs

Preparation

- Unified input–output format
- Cleaning and normalization aligned with inference

Splitting

- Train / Validation / Test
- Split before statistical fitting to prevent leakage

Candidate Selection & Data Controls

Include

- Complete original–rewrite pairs
- Explicit tone selection
- Finalized user messages

Exclude

- Empty, spam, bot, or incomplete records
- Synthetic QA traffic
- Data without user consent

Quality Controls

- Deduplication and near-duplicate removal
- Decontamination to prevent evaluation overlap

Leakage Prevention & Evaluation Strategy

Leakage Prevention

- Split before tokenization and vocabulary fitting
- No duplicate pairs across splits
- No features derived from rewritten outputs

Chronological Evaluation

- Train on historical window
- Hold out recent data for testing
- Preserve conversation-level integrity

DevOps: Retraining & Deployment Strategy

- Weekly retraining of classifier and generator models
- Conditional retraining triggered by:
  - Data drift detection
  - Drop in suggestion acceptance rate

- Automated CI/CD pipeline manages:
  - Data validation
  - Model training & evaluation
  - Deployment

- Models must meet performance criteria ($F1 \geq 0.85$)
- Canary deployment before full rollout

Scaling & Infrastructure Design

- Typical load: ~5 requests/second
- Peak load: ~20 requests/second

- Generator runs on 1 GPU instance
- Classifier runs on CPU-based pod

- Autoscaling enabled if CPU usage > 70%
- Suitable for university-scale deployment
- Infrastructure expandable if usage increases

Success Metrics & Risk Mitigation

Success Metrics:

- Acceptance rate ≥ 35%
- Undo rate < 5%
- Increase in appropriate tone usage

Failure Risks:

- Over-formalization
- Tone misclassification
- Drift in communication patterns
- Inappropriate rewrites

Mitigation:

- Confidence thresholds
- Toxicity filtering
- Human-in-the-loop approval
- Continuous monitoring